Vectors and Vector Spaces

80/20 Applied Mathematics of Machine Learning — Interview Notes

---

1. Why Vectors Matter in Machine Learning

Core idea

Machine learning works with data, and data must be represented mathematically.

A single data sample can be represented as a **vector**.

For the Iris dataset:

```
                    Features
                ┌──────────────────────────┐
Sample 1  →     | 5.1   3.5   1.4   0.2    |
                └──────────────────────────┘
                   |     |     |     |
                   ↓     ↓     ↓     ↓
                sepal sepal petal petal
                length width length width
```

Therefore:

```
x = (5.1, 3.5, 1.4, 0.2)
```

is one data vector.

The entire dataset becomes a matrix:

```
              Features
           f1   f2   f3   f4
         ┌──────────────────────┐
sample1  | 5.1  3.5  1.4  0.2 |
sample2  | 4.9  3.0  1.4  0.2 |
sample3  | 4.7  3.2  1.3  0.2 |
  ...    | ...  ...  ...  ... |
sample150
         └──────────────────────┘
```

```
X ∈ R^(150×4)
```

Interview takeaway

> A vector provides a compact mathematical representation of related measurements, and matrices allow us to represent collections of vectors.

---

2. Vector — Definition

A vector can be represented as an ordered tuple:

```
x = (x₁, x₂, ..., xₙ)
```

For example:

```
x = (2, 5, -1)
```

In ML, the components usually represent **features**.

Example:

```
x = (age, income, experience)
```

or

```
x = (sepal_length, sepal_width, petal_length, petal_width)
```

Important conceptual point

A vector is **not fundamentally defined by being an arrow**.

The arrow is a useful geometric interpretation.

The deeper idea is that vectors belong to a mathematical structure where we can perform operations such as:

```
vector + vector
scalar × vector
```

---

3. Vector Addition

Given:

```
x = (x₁, x₂, ..., xₙ)
y = (y₁, y₂, ..., yₙ)
```

their sum is:

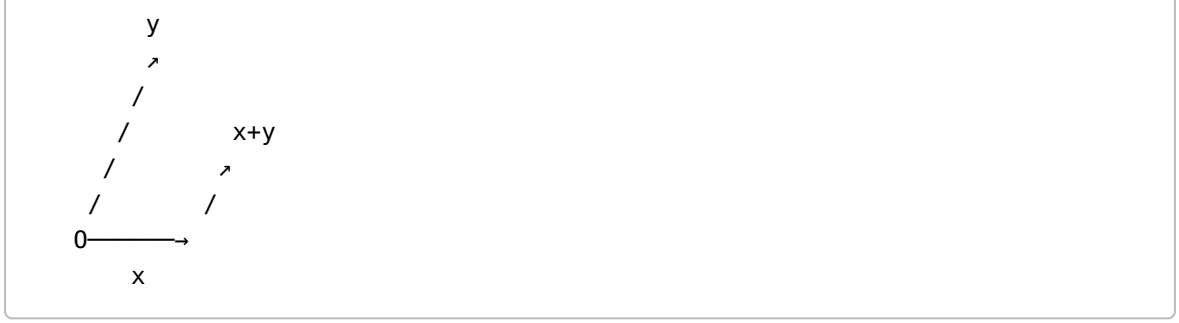
```
x + y =
(x₁ + y₁, x₂ + y₂, ..., xₙ + yₙ)
```

Example

```
x = (1, 2)
y = (3, 4)

x + y = (4, 6)
```

Geometric interpretation



Vector addition can be understood as **translation**.

---

4. Scalar Multiplication

A scalar is simply a number.

For:

```
c = 3
x = (1, 2)
```

we get:

```
3x = (3, 6)
```

Generally:

```
cx = (cx₁, cx₂, ..., cxₙ)
```

Geometric interpretation

```
x       = ————————→

2x      = ————————————————→

0.5x    = ————→

-x      = ←————————
```

- `|c| > 1` → stretching
- `0 < |c| < 1` → shrinking
- `c < 0` → direction reversal
- `c = 0` → zero vector

---

5. What Is a Vector Space?

Definition

A vector space is a set of vectors together with:

1. Vector addition
2. Scalar multiplication

such that the required algebraic rules are satisfied.

We commonly work with:

```
Rⁿ
```

where the scalars are real numbers.

Examples:

```
R²
R³
R⁴
```

```
...
Rⁿ
```

But vector spaces can also contain:

- polynomial functions
- continuous functions
- other mathematical objects

80/20 interview definition

> A vector space is a set of objects that is closed under vector addition and scalar multiplication and satisfies the standard vector-space axioms.

---

6. Vector Space Axioms — What You Actually Need

You do not need to blindly memorize every axiom. Understand the structure.

For vectors `x, y, z` and scalars `a, b`:

Addition

```
x + y = y + x
```

Commutativity.

```
(x + y) + z = x + (y + z)
```

Associativity.

There exists a zero vector:

```
x + 0 = x
```

Every vector has an additive inverse:

```
x + (-x) = 0
```

Scalar multiplication

```
a(bx) = (ab)x
```

```
a(x + y) = ax + ay
```

```
(a + b)x = ax + bx
```

```
1x = x
```

Interview insight

The important idea is:

> Vector spaces provide a consistent algebraic environment in which vectors can be added and scaled.

---

7. Why Abstract Vector Spaces?

This is an important conceptual interview question.

A vector does not have to literally be an array.

For example:

```
(2, 3, 4)
```

is a vector.

But a polynomial can also be treated as a vector:

```
p(x) = 2 + 3x + 4x²
```

which can be represented by its coefficients:

```
(2, 3, 4)
```

Similarly, functions can form vector spaces.

Key insight

The abstraction lets us apply the same mathematical reasoning to very different objects.

```
Vectors
   │
   ├── numerical arrays
   ├── polynomials
   ├── functions
   └── other mathematical objects
```

Interview answer

> The abstraction separates the mathematical structure from the physical representation. This lets us reason about arrays, functions, polynomials, etc. using the same concepts such as linear combination, basis, span, and linear transformation.

---

8. Linear Combination ⭐ P0

This is one of the most important concepts.

Given vectors:

$$v_1, v_2, ..., v_n$$

a linear combination is:

$$c_1v_1 + c_2v_2 + ... + c_nv_n$$

where the c_i are scalars.

Example

$$v_1 = (1, 0)$$
$$v_2 = (0, 1)$$

Then:

$$3v_1 + 5v_2$$

$$= 3(1,0) + 5(0,1)$$

```
= (3,5)
```

Therefore `(3,5)` is a linear combination of v_1 and v_2.

Mental model

```
Basis vectors
     ↓
scale them
     ↓
add them
     ↓
obtain another vector
```

This concept is the foundation for:

- span
- basis
- linear independence
- matrix multiplication
- linear transformations
- dimensionality reduction
- regression
- neural networks

---

9. Linear Independence ⭐ P0

Definition

Vectors are linearly independent if the only way to obtain the zero vector from their linear combination is by using all zero coefficients.

Mathematically:

$$c_1v_1 + c_2v_2 + ... + c_nv_n = 0$$

implies:

$$c_1 = c_2 = ... = c_n = 0$$

Example — Independent

```
v₁ = (1,0)
v₂ = (0,1)
```

The equation:

```
c₁(1,0) + c₂(0,1) = (0,0)
```

gives:

```
c₁ = 0
c₂ = 0
```

Therefore they are independent.

---

10. Linear Dependence

Consider:

```
v₁ = (1,0)
v₂ = (0,1)
v₃ = (1,1)
```

Notice:

```
v₃ = v₁ + v₂
```

Therefore:

```
v₁ + v₂ - v₃ = 0
```

with non-zero coefficients.

Therefore the vectors are **linearly dependent**.

Visual

```
v₁ ──────→

            \──────→ v₃
```

```
            /
v₂ ————→
```

v_3 contains information already provided by v_1 and v_2.

Mental model

> Linear dependence = redundancy.
>
> Linear independence = no vector can be reconstructed from the others.

---

11. Fast Interview Test for Independence

Given vectors:

```
v₁, v₂, ..., vₙ
```

ask:

> Can one vector be constructed as a linear combination of the others?

If yes:

```
DEPENDENT
```

If no:

```
INDEPENDENT
```

For matrices, a practical computational method is to use **rank**.

```
number of independent vectors = rank
```

For a set of vectors:

```
rank = number of linearly independent vectors
```

---

12. Span ⭐ P0

Definition

The span of a set of vectors is the set of **all possible linear combinations** of those vectors.

For:

$$S = \{v_1, v_2\}$$

we write:

$$\text{span}(v_1, v_2)$$

and it contains:

$$a\ v_1 + b\ v_2$$

for all possible scalars a and b.

Example

$$v_1 = (1,0)$$
$$v_2 = (0,1)$$

Then:

$$\text{span}(v_1,v_2) = R^2$$

because every (x,y) can be written:

$$(x,y) = x(1,0) + y(0,1)$$

---

13. Geometric Meaning of Span

One non-zero vector

The span is a line.



Two independent vectors in R^3

The span is a plane.



Three independent vectors in R^3

The span is all of:

```
R³
```

Important pattern

```
1 independent vector → line
2 independent vectors → plane
3 independent vectors → 3D space
```

assuming the vectors live in `R³`.

---

14. Generating Set

A set `S` generates `V` if:

```
span(S) = V
```

Example:

```
{(1,0), (0,1)}
```

generates `R²`.

Because:

```
(x,y) = x(1,0) + y(0,1)
```

---

15. Basis ⭐⭐⭐ P0

This is one of the most important concepts in the chapter.

Definition

A basis of a vector space must satisfy **two conditions**:

```
1. Linearly independent
2. Spans the entire vector space
```

Therefore:

```
BASIS
  =
INDEPENDENT
  +
SPANNING
```

Mental model

A basis is the **minimum set of building blocks needed to construct every vector in the space**.

---

16. Standard Basis

For `R²`:

```
e₁ = (1,0)
e₂ = (0,1)
```

For `R³`:

```
e₁ = (1,0,0)
e₂ = (0,1,0)
e₃ = (0,0,1)
```

For R^n:

```
e₁ = (1,0,...,0)
e₂ = (0,1,...,0)
...
eₙ = (0,0,...,1)
```

Every vector can be represented using these basis vectors.

Example:

```
x = (4,7)

  = 4(1,0) + 7(0,1)
```

---

17. Basis as a Coordinate System ⭐ P0

This is a very important conceptual distinction.

A vector and its coordinates are **not necessarily the same thing**.

Suppose:

```
v₁ = (1,1)
v₂ = (1,-1)
```

These can form a basis.

A vector might be represented as:

```
x = 3v₁ + 2v₂
```

Its coordinates relative to this basis are:

```
(3,2)
```

But relative to the standard basis, its coordinates are different.

Key insight

```
Same vector
     │
     ├── coordinates in Basis A
     │
     └── coordinates in Basis B
```

The **vector is the mathematical object**.

Coordinates are its representation relative to a chosen basis.

This becomes extremely important when studying:

- matrices
- linear transformations
- change of basis
- eigenvectors
- PCA
- coordinate systems

---

18. Why Basis Is Important in Machine Learning

A basis provides a compact representation.

Instead of describing every vector individually:

```
x₁
x₂
x₃
...
```

we describe vectors using coefficients relative to basis vectors:

```
x = c₁v₁ + c₂v₂ + ... + cₙvₙ
```

This idea appears throughout ML.

For example:

```
Data
 ↓
Representation
 ↓
```

```
Basis / coordinates
 ↓
Transformations
 ↓
Prediction
```

---

19. Dimension ⭐ P0

The dimension of a finite-dimensional vector space is the number of vectors in any basis.

For:

```
R²
```

dimension:

```
dim(R²) = 2
```

For:

```
R³
```

dimension:

```
dim(R³) = 3
```

For:

```
Rⁿ
```

dimension:

```
dim(Rⁿ) = n
```

Important theorem

Every basis of the same finite-dimensional vector space has the same number of vectors.

Therefore dimension is well-defined.

---

20. Finite-Dimensional vs Infinite-Dimensional

Finite-dimensional

A vector space has a finite basis.

Example:

$$R^3$$

Basis:

$$\{(1,0,0),(0,1,0),(0,0,1)\}$$

Dimension:

$$3$$

Infinite-dimensional

Some vector spaces require infinitely many basis vectors.

Example: polynomial space.

Basis:

$$\{1, x, x^2, x^3, ...\}$$

There is no finite basis containing all polynomial degrees.

ML relevance

Most practical ML computations operate in finite-dimensional spaces such as:

$$R^n$$

---

21. Isomorphism — Understand, Don't Memorize ⭐ P1

Every finite-dimensional real vector space of dimension `n` has essentially the same structure as:

```
Rⁿ
```

This relationship is described by an **isomorphism**.

Mental model

```
Abstract vector space V
          |
          | isomorphism
          ↓
         Rⁿ
```

The representation changes, but the vector-space structure is preserved.

Interview answer

> An isomorphism is a structure-preserving one-to-one correspondence between vector spaces. It means the spaces may look different but have the same underlying vector-space structure.

---

22. Subspace ⭐ P0

A subspace is a subset of a vector space that is itself a vector space.

For a set `U ⊆ V` to be a subspace, it must be closed under:

```
vector addition
scalar multiplication
```

Example

A line through the origin in `R²` is a subspace.

```
        /
       /
      /
-----0---------
    /
   /
```

A line that does **not** pass through the origin is not a subspace.

```
      /
     /
----/---------
```

Critical interview test

A subset `U` is a subspace if:

```
0 ∈ U
```

and for:

```
u,v ∈ U
a,b ∈ R
```

we have:

```
au + bv ∈ U
```

This single condition captures closure under linear combinations.

---

23. Span Is Always a Subspace

For any set of vectors:

```
S
```

the span:

```
span(S)
```

is a subspace.

Why?

Because linear combinations remain linear combinations.

If:

```
u ∈ span(S)
v ∈ span(S)
```

then:

```
au + bv
```

is also in:

```
span(S)
```

---

24. Direct Sum

If two subspaces U_1 and U_2 combine to form another space:

```
U₁ + U₂
```

then elements look like:

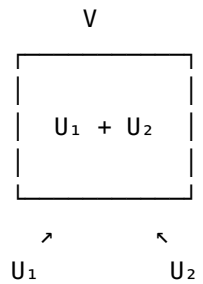
```
u₁ + u₂
```

where:

```
u₁ ∈ U₁
u₂ ∈ U₂
```

The direct-sum idea is useful for understanding how a larger space can be constructed from smaller subspaces.

Mental picture



ML relevance

This becomes more important later for:

- matrix decompositions
- orthogonal decompositions
- projections
- linear transformations

For a first interview pass, understand the concept rather than memorizing the theorem proof.

---

25. Scaling Data — Major ML Application ⭐⭐⭐

This is one of the most practically important sections.

Suppose we have:

```
Feature A: 1 – 10
Feature B: 1,000 – 10,000
```

A model using these raw features may allow Feature B to dominate numerical calculations.

Analogy

```
Feature A = whisper
Feature B = loud music

Model
  ↓
mostly hears Feature B
```

Scaling puts features on comparable numerical scales.

---

26. Standardization

The common transformation is:

```
x_scaled = (x - μ) / σ
```

where:

```
μ = mean
σ = standard deviation
```

For a vector:

```
x = (x₁,x₂,...,xₙ)
```

the operation is applied feature-wise.

Mean

For `m` samples:

```
μ = (1/m) Σ xᵢ
```

Standard deviation

```
σ = sqrt((1/m) Σ(xᵢ - μ)²)
```

Result

Each feature is transformed approximately to:

```
mean = 0
standard deviation = 1
```

---

27. Why Feature Scaling Matters

Scaling can significantly affect algorithms that depend on:

- distance
- magnitude
- gradients
- optimization

Examples:

```
KNN
K-means
SVM
PCA
```

```
gradient-based optimization
neural networks
```

Important nuance

Not every algorithm is equally sensitive to scaling.

Tree-based models such as:

```
Decision Trees
Random Forests
Gradient-boosted trees
```

are generally much less sensitive to feature scaling.

Interview-quality answer

> Feature scaling is important when an algorithm depends on feature magnitude, distance, or optimization geometry. Standardization transforms features using `(x-μ)/σ`, giving them a comparable scale.

---

28. NumPy Implementation ⭐⭐⭐

For ML engineering, this implementation is important.

```
import numpy as np

X_scaled = (X - X.mean(axis=0)) / X.std(axis=0)
```

If:

```
X.shape = (150, 4)
```

then:

```
X.mean(axis=0)
```

produces:

```
4 feature means
```

and:

```
X.std(axis=0)
```

produces:

```
4 feature standard deviations
```

The result is still:

```
(150, 4)
```

---

29. Why axis=0?

Suppose:

```
X =
[
 [a,b,c],
 [d,e,f],
 [g,h,i]
]
```

Then:

```
X.mean(axis=0)
```

means:

```
column-wise mean
```

Result:

```
[
 mean(a,d,g),
 mean(b,e,h),
 mean(c,f,i)
]
```

So:

```
axis=0 → operate down rows → feature-wise
axis=1 → operate across columns → sample-wise
```

Interview trap

Do not simply memorize `axis=0`.

Understand what dimension is being reduced.

---

30. Python List vs NumPy Array ⭐⭐⭐

This is an important implementation concept.

Python List

```
x = [1, 2, 3]
```

Lists are:

- flexible
- dynamic
- heterogeneous
- easy to use
- not optimized for numerical vector operations

Example:

```
[1, 2, 3] + [4, 5, 6]
```

produces:

```
[1, 2, 3, 4, 5, 6]
```

This is concatenation, not vector addition.

---

31. NumPy Array

```
x = np.array([1, 2, 3])
```

Now:

```
x + np.array([4,5,6])
```

produces:

```
[5,7,9]
```

which matches vector addition.

Scalar multiplication:

```
3 * x
```

produces:

```
[3,6,9]
```

---

32. Why NumPy Is Faster ⭐⭐⭐

NumPy arrays are designed for numerical computation.

Three major reasons:

```
1. Homogeneous data type
2. Contiguous memory
3. Operations implemented in optimized native code
```

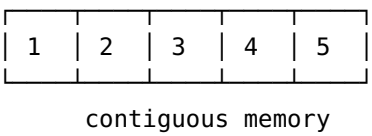
Python list

```
List
 │
 ├── pointer → object
 ├── pointer → object
 ├── pointer → object
 └── pointer → object

Objects may be scattered in memory
```

NumPy array

```
NumPy array
```



This improves:

- cache locality
- memory efficiency
- vectorized computation
- throughput

---

33. Vectorization ⭐⭐⭐

Instead of:

```
result = []

for x in values:
    result.append(3*x**2 - x**4)
```

NumPy allows:

```
result = 3 * X**2 - X**4
```

The operation is performed over the whole array.

Mental model

```
Python loop

Python
 ↓
element
 ↓
Python
 ↓
element
 ↓
Python
 ↓
element
 ↓
...
```

```
versus

NumPy

Python
  ↓
large array operation
  ↓
optimized native implementation
```

This is called **vectorization**.

---

34. Vectorization Is Not Just "Using Vectors"

Very important interview distinction.

Mathematical vector:

```
x = [1,2,3]
```

Vectorized computation:

```
3 * x**2 - x**4
```

means performing the operation over the entire array using optimized array operations rather than explicit Python-level loops.

Why it matters

It reduces Python interpreter overhead and makes better use of optimized native numerical routines.

---

35. NumPy Performance — Important Nuance

NumPy is not automatically faster for every operation.

For one tiny operation:

```
random_np()
```

the overhead of entering NumPy can make it slower than:

```
random_py()
```

But for large batches:

```
random_np(1000)
```

NumPy can generate the entire array efficiently.

General rule

```
Small operation
     ↓
Python may be competitive

Large numerical operation
     ↓
NumPy usually wins
```

Interview answer

> NumPy's advantage comes from performing bulk numerical operations efficiently, not from every individual NumPy function call being faster than Python.

---

36. Array Memory Model ⭐⭐

Computers store data as bits.

```
bit → 0 or 1
```

Groups of bits represent values.

Examples:

```
int32
float32
float64
```

A NumPy array typically stores elements of the same data type in a structured memory region.

Example:

```
float64 array

┌─────────┬─────────┬─────────┬─────────┐
│ 8 byte  │ 8 byte  │ 8 byte  │ 8 byte  │
└─────────┴─────────┴─────────┴─────────┘
```

This predictable layout makes numerical operations efficient.

---

37. dtype

NumPy arrays have a data type:

```
X.dtype
```

Example:

```
dtype('float64')
```

All elements generally share this dtype.

Therefore:

```
X[0] = 23
```

can become:

```
23.0
```

because it can be represented as `float64`.

But:

```
X[0] = "str"
```

fails because a string cannot be converted to the array's numeric dtype in that context.

---

38. Tuples vs Lists vs NumPy Arrays

Property	Tuple	List	NumPy Array
Mutable	No	Yes	Yes
Dynamic size	No	Yes	No
Mixed types	Yes	Yes	Usually no
Vector arithmetic	No	No	Yes
Numerical performance	Low	Low	High
Scientific computing	Poor	Poor	Excellent
Memory efficiency	Better than flexible list in some cases	Lower	High for numeric data

Interview takeaway

> Python lists are general-purpose containers; NumPy arrays are specialized numerical data structures.

---

39. Immutable Tuple

Example:

```
v = (1, 2, 3)
```

Cannot do:

```
v[0] = 10
```

because tuples are immutable.

Benefit

Immutability can prevent accidental modification.

Cost

A new object is needed for changes.

For large numerical computations, NumPy arrays are generally more suitable.

---

40. Python List Memory Model

Conceptually:

```
Python list

┌─────────┬─────────┬─────────┐
│ pointer│ pointer│ pointer│
└────┬────┴────┬────┴────┬────┘
     ↓         ↓         ↓
 object    object    object
```

The objects may be located in different memory locations.

This reduces memory locality compared with a homogeneous contiguous numerical array.

---

41. NumPy's Main Advantage

The important comparison is:

```
Python List
     ↓
flexibility
     ↓
higher overhead

NumPy ndarray
     ↓
less flexibility
     ↓
high numerical performance
```

This is a classic engineering trade-off:

> Flexibility vs performance.

---

42. Important NumPy Operations

Create an array

```
x = np.array([1, 2, 3])
```

Zeros

```
np.zeros(5)
```

Ones

```
np.ones(5)
```

Random values

```
np.random.rand(5)
```

Shape

```
X.shape
```

Data type

```
X.dtype
```

Slice

```
X[1:4]
```

Element access

```
X[0]
```

Element-wise multiplication

```
x * y
```

---

43. Important Distinction: Element-wise Multiplication

For:

```
x = (1,2,3)
y = (4,5,6)
```

NumPy:

```
x * y
```

means:

```
(1×4, 2×5, 3×6)

= (4,10,18)
```

This is **element-wise multiplication**.

It is not the same thing as the mathematical matrix/vector operations you will later study, such as:

```
dot product
matrix multiplication
```

Interview warning

Do not confuse:

```
element-wise multiplication
```

with:

```
dot product
```

---

44. Dot Product — Preview

Although it is developed later, understand the basic idea.

For:

```
x = (x₁,x₂,x₃)
y = (y₁,y₂,y₃)
```

the dot product is:

$$x \cdot y = x_1y_1 + x_2y_2 + x_3y_3$$

Example:

$$(1,2,3) \cdot (4,5,6)$$

$$= 1\times4 + 2\times5 + 3\times6$$

$$= 32$$

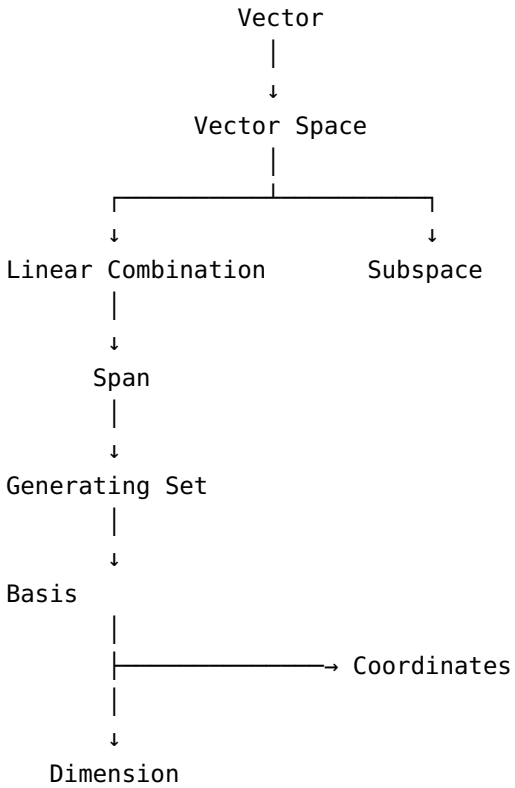
This becomes fundamental for:

- linear regression
- neural networks
- similarity
- projections
- embeddings
- attention

---

45. The Big Picture

The concepts form a hierarchy:



And:

```
Vector
   ↓
Operations
   ↓
Linear combinations
   ↓
Span
   ↓
Basis
   ↓
Coordinates
   ↓
Linear transformations
   ↓
Matrices
   ↓
Machine Learning Models
```

This is the conceptual chain you should remember.

---

46. 80/20 Priority Classification

P0 — MUST MASTER

These are interview-critical.

Mathematical

- Vector
- Vector addition
- Scalar multiplication
- Vector space intuition
- Linear combination
- Linear independence
- Linear dependence
- Span
- Generating set
- Basis
- Standard basis
- Coordinates
- Dimension
- Subspace

ML application

- Feature vectors

- Dataset as matrix
- Feature scaling
- Mean
- Standard deviation
- Standardization

Implementation

- NumPy ndarray
- `shape`
- `dtype`
- indexing
- slicing
- vectorized operations
- element-wise operations
- `axis=0`
- basic memory/layout intuition
- Python list vs NumPy array

---

47. P1 — IMPORTANT AFTER P0

Understand these after mastering the above:

- Abstract vector spaces
- Polynomial vector spaces
- Function spaces
- Isomorphism
- Direct sums
- Maximal linearly independent sets
- Minimal generating sets
- Basis existence theorem
- Computational performance profiling
- cache locality
- Python interpreter overhead

These are useful for deeper mathematical understanding and senior-level technical discussions, but they should not delay mastery of P0.

---

48. P2 — LOW PRIORITY FOR ML INTERVIEWS

Do not spend disproportionate time here initially.

- Transfinite induction proof of basis existence
- Detailed set-theoretic construction
- Formal proof of every vector-space axiom
- Historical discussion of vector-space abstraction
- Deep implementation details of CPython internals

- Exact benchmark numbers from the textbook

Understand the ideas, but don't memorize them.

---

49. What Requires Implementation Practice?

Definitely implement ⭐⭐⭐

Task 1 — Vector operations

```
import numpy as np

x = np.array([1, 2, 3])
y = np.array([4, 5, 6])

print(x + y)
print(3 * x)
print(x * y)
```

Expected:

```
x + y → [5,7,9]
3x   → [3,6,9]
x*y  → [4,10,18]
```

Task 2 — Standardization

```
X = np.array([
    [10, 100],
    [20, 200],
    [30, 300]
])

mean = X.mean(axis=0)
std = X.std(axis=0)

X_scaled = (X - mean) / std

print(X_scaled)
```

Understand exactly what happens to each column.

Task 3 — Verify the result

```
print(X_scaled.mean(axis=0))
print(X_scaled.std(axis=0))
```

You should obtain approximately:

```
mean → [0, 0]
std  → [1, 1]
```

Task 4 — Demonstrate list vs NumPy

```
a = [1, 2, 3]
b = [4, 5, 6]

print(a + b)
```

versus:

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

print(a + b)
```

Understand why the results differ.

---

50. Interview-Level Example: Basis

Question

Is:

```
{(1,0), (0,1)}
```

a basis for `R²`?

Answer

Yes.

Reason 1:

They are linearly independent.

Reason 2:

They span R^2.

Every vector:

```
(x,y)
```

can be written as:

```
x(1,0) + y(0,1)
```

Therefore:

```
{(1,0),(0,1)}
```

is a basis.

---

51. Interview-Level Example: Not a Basis

Consider:

```
v₁ = (1,0)
v₂ = (2,0)
```

They are linearly dependent because:

```
v₂ = 2v₁
```

They also only span a line.

Therefore they are **not a basis for R^2**.

They could be a generating set for that particular line, but not for all of R^2.

---

52. Interview-Level Example: Redundant Vector

Consider:

```
v₁ = (1,0)
v₂ = (0,1)
v₃ = (1,1)
```

Since:

```
v₃ = v₁ + v₂
```

`v₃` is redundant.

Therefore:

```
{v₁,v₂,v₃}
```

is linearly dependent.

But:

```
{v₁,v₂}
```

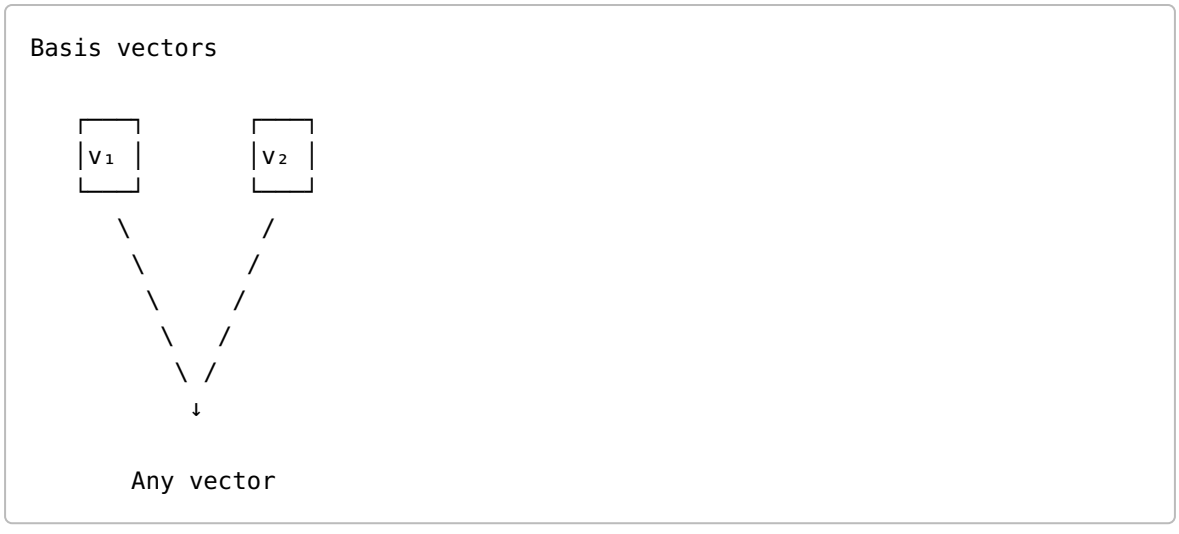
still spans `R²`.

This demonstrates why a basis removes redundancy.

---

53. Basis Mental Model

Think of a basis as LEGO blocks.



A basis has exactly the right amount of information:

```
Too few vectors
      ↓
cannot represent everything

Too many redundant vectors
      ↓
not linearly independent

Basis
      ↓
complete + non-redundant
```

---

54. Basis = Minimal + Complete

This is an excellent interview phrase:

> A basis is simultaneously a minimal generating set and a maximal linearly independent set.

Meaning:

Remove a basis vector

```
→ lose representational capability
```

Add another vector

```
→ introduce redundancy
```

Therefore:

```
Basis
 = complete
 + non-redundant
```

---

55. Scaling Mental Model

Suppose:

```
Age       = 20–60
Salary    = 20,000–2,00,000
```

Without scaling:

```
Salary magnitude
       ████████████████
Age    ███
```

After standardization:

```
Age       → comparable scale
Salary    → comparable scale
```

The goal is not to change the underlying information.

It changes the **representation** so algorithms can work more effectively.

---

56. Machine Learning Connection

A useful high-level mental model is:

```
Raw data
   ↓
Vectors
   ↓
Matrices
   ↓
Vector operations
   ↓
Linear transformations
   ↓
Feature representations
   ↓
Model
   ↓
Prediction
```

For example, a neural network layer can be viewed as a transformation involving vectors and matrices:

```
input vector
     x
     |
```

```
    ↓
  Matrix
    W
    |
    ↓
   Wx
    |
    ↓
transformed
 vector
```

Understanding vectors is therefore foundational to understanding neural networks.

---

57. Why the Chapter Says "Machine Learning Is Data Transformation"

A useful conceptual view is:

```
Raw data
   ↓
Cleaning
   ↓
Scaling
   ↓
Feature transformation
   ↓
Representation
   ↓
Prediction
```

Many ML algorithms transform data into representations where prediction becomes easier.

Examples:

```
standardization
PCA
embeddings
neural network layers
feature engineering
```

All rely heavily on vector/matrix operations.

---

58. Common Interview Traps

Trap 1

> Is every tuple a vector?

Not automatically.

A mathematical vector is an element of a vector space. A tuple is one possible representation of a vector.

Trap 2

> Is a vector defined by magnitude and direction?

Not fundamentally.

That is a useful geometric interpretation, but abstract vectors can be polynomials, functions, etc.

Trap 3

> Is every set of vectors a basis?

No.

A basis must:

```
span the space
AND
be linearly independent
```

Trap 4

> Does spanning imply independence?

No.

Example:

```
(1,0)
(0,1)
(1,1)
```

These span `R²`, but they are dependent.

Trap 5

> Does independence imply spanning?

No.

A single non-zero vector in `R²` is independent but doesn't span all of `R²`.

Trap 6

> Is NumPy always faster than Python?

No.

NumPy's advantage is especially strong for **large/batched numerical operations**.

Trap 7

> Is `x * y` the dot product in NumPy?

No.

For arrays:

```
x * y
```

is element-wise multiplication.

Dot product requires something like:

```
np.dot(x, y)
```

or:

```
x @ y
```

for appropriate dimensions.

---

59. Most Important Formulas

Vector addition

```
x + y =
(x₁+y₁, ..., xₙ+yₙ)
```

Scalar multiplication

```
cx =
(cx₁, ..., cxₙ)
```

Linear combination

```
Σ cᵢvᵢ
```

Linear independence

```
Σ cᵢvᵢ = 0
```

only when:

```
all cᵢ = 0
```

Mean

```
μ = (1/m) Σxᵢ
```

Standard deviation

```
σ = sqrt((1/m)Σ(xᵢ-μ)²)
```

Standardization

```
x_scaled = (x-μ)/σ
```

Dimension

```
dim(V) = number of vectors in a basis
```

60. One-Page Interview Cheat Sheet

```
VECTOR
An element that participates in vector-space operations.

VECTOR SPACE
A mathematical structure closed under vector addition
and scalar multiplication with the required axioms.

LINEAR COMBINATION
c₁v₁ + c₂v₂ + ... + cₙvₙ

LINEAR INDEPENDENCE
Σcᵢvᵢ = 0 ⇒ all cᵢ = 0

DEPENDENCE
At least one non-zero coefficient can produce zero.

SPAN
All possible linear combinations of a set.

GENERATING SET
A set whose span equals the entire vector space.

BASIS
Linearly independent + spans the space.

STANDARD BASIS OF Rⁿ
e₁, e₂, ..., eₙ

DIMENSION
Number of vectors in a basis.

SUBSPACE
A subset that is itself a vector space.

ISOMORPHISM
Structure-preserving correspondence.

FEATURE VECTOR
One sample represented as a vector.

DATASET
Usually represented as a matrix.

STANDARDIZATION
(x - μ) / σ
```

```
NUMPY
Efficient numerical array computation.

VECTORIZATION
Performing bulk operations on arrays instead
of explicit Python-level loops.
```

---

61. Interview Questions You Should Be Able to Answer

After studying this section, you should be able to answer these without notes:

Fundamentals

1. What is a vector?
2. What is a vector space?
3. Why do we need vector spaces?
4. What is vector addition?
5. What is scalar multiplication?
6. What is a linear combination?

Basis / structure

1. What is linear independence?
2. What is linear dependence?
3. What is span?
4. What is a generating set?
5. What is a basis?
6. Why must a basis be linearly independent?
7. Why must a basis span the space?
8. What is the standard basis?
9. What is dimension?
10. Can a vector space have multiple bases?
11. Can the same vector have different coordinates?

ML

1. Why are data points represented as vectors?
2. Why do we scale features?
3. What is standardization?
4. Why does feature scaling matter?
5. Which ML algorithms are sensitive to feature scaling?

NumPy

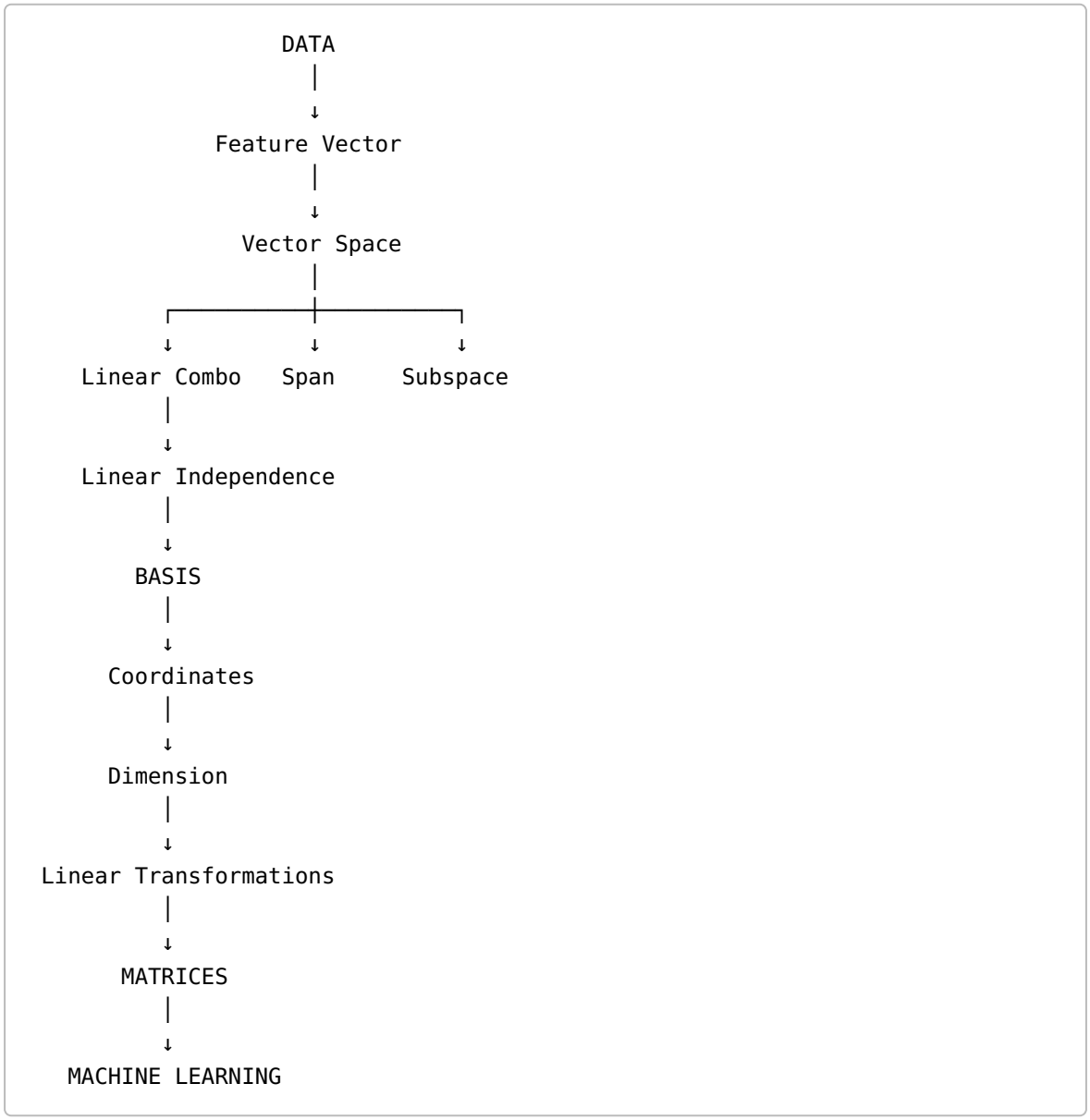
1. Why use NumPy arrays instead of Python lists?
2. Why is NumPy faster for large numerical computations?
3. What is vectorization?

4. What does `axis=0` mean?
5. What is `dtype`?
6. What is the difference between `x*y` and a dot product?
7. Why isn't NumPy always faster than Python?

---

62. Final 80/20 Mental Model

If you remember only one diagram, remember this:



And for implementation:

```
Python List
    ↓
Flexible but inefficient for numerical workloads
```

```
NumPy ndarray
    ↓
Homogeneous + contiguous + optimized
    ↓
Vectorized computation
    ↓
Efficient ML data processing
```

The **highest-value concepts for interviews** are:

```
Vector
   ↓
Linear combination
   ↓
Linear independence
   ↓
Span
   ↓
Basis
   ↓
Dimension
   ↓
Subspace
   ↓
NumPy representation
   ↓
Feature scaling
   ↓
Vectorization
```

Master these deeply before spending time on the more abstract proofs.